FINAL YEAR PROJECT REPORT

SMARTBIZ

A Mobile-First Business Management System for Small Businesses in Nepal

Prepared and submitted by

SANSKAR KAYASTHA ([STUDENT ID])

8th Semester

In partial fulfilment of the requirements for the degree of

BACHELOR OF INFORMATION AND COMMUNICATION TECHNOLOGY

School of Science and Technology
Virinchi College
Asia e University

JULY 2026

DECLARATION

I hereby declare that the project entitled “SmartBiz: A Mobile-First Business Management System for Small Businesses in Nepal” is my original work carried out in partial fulfilment of the requirements for the Bachelor of Information and Communication Technology degree. The report has not been submitted, in whole or in part, for any other academic award.

All sources of information used in this work have been acknowledged in the reference section. The design, implementation, testing evidence and analysis presented here are based on the SmartBiz repository and the development work completed during the project period. Where third-party frameworks, hosted services and documentation were used, their contribution has been identified appropriately.

Declaration signature record

Candidate	Signature	Date
Sanskar Kayastha	_____	_____

APPROVED BY

This final year project report has been examined and approved as meeting the academic requirements of the Bachelor of Information and Communication Technology programme.

Project approval record

Name and role	Signature	Date
Chiranjibi Shrestha Project Supervisor		
Binay Malla Academic Coordinator		
Rastra Bhushan Khadka Principal		

ACKNOWLEDGEMENTS

The completion of SmartBiz was made possible through the guidance, encouragement and practical support of several individuals. I express my sincere gratitude to my project supervisor, Chiranjibi Shrestha, for providing direction throughout the analysis, implementation and documentation stages. His feedback helped transform a broad business-management idea into a focused, testable system.

I am thankful to the faculty members and administration of Virinchi College and Asia e University for providing the academic framework and resources required for this project. I also appreciate the small-business owners, classmates and peers whose observations about daily stock, sales and customer-management difficulties informed the system requirements.

Finally, I acknowledge my family and friends for their patience and continuous encouragement during the twelve-week development period. Their support was especially valuable while integrating multiple services, testing mobile and web workflows, and preparing the final report.

ABSTRACT

Small businesses in Nepal frequently manage products, suppliers, sales, customer credit and leads through notebooks or disconnected spreadsheets. These practices are familiar and inexpensive, but they create duplicate entry, delayed stock updates, limited business visibility and a higher risk of losing important records. SmartBiz was developed as a mobile-first business management system that brings these activities into one authenticated platform while retaining a simple workflow suitable for a small team.

The system uses a Java 21 and Spring Boot 3.4.5 microservice backend, PostgreSQL databases, a Spring Cloud Gateway, Eureka service discovery, Redis caching, a React Native application built with Expo, and a Next.js web dashboard. Its core functions include authentication and billing, inventory and supplier management, point-of-sale recording, sales analytics, customer and lead management, invoice scanning, voice-assisted entry, AI-generated insight cards and controlled sales-file imports. Each service owns its database and communicates with other services through REST APIs. A signed JSON Web Token identifies the user, while the gateway propagates an X-User-Id value so that business records remain isolated by tenant.

The most critical workflow is sale recording. For online payments, SmartBiz reserves stock before checkout, verifies the provider result, and commits the reservation only after successful payment confirmation. Failed, cancelled or expired payments release the reservation and do not deduct stock. This reduces overselling risk without introducing direct cross-service database access. Flyway migrations, DTO-based responses, containerised deployment and lenient Redis error handling support maintainability and recovery.

Repository-level automated tests were executed with the required Java 21 runtime. Fifty-seven of fifty-eight discovered tests passed; one AI insight-card expectation remains a documented quality issue because the generated card set omitted a RESTOCK_SOON signal for the test fixture while returning the other expected business signals. The evaluation therefore concludes that the MVP is functionally comprehensive and architecturally consistent, while production readiness would benefit from resolving that test, formal load testing, refresh-token rotation, stronger observability and expansion of the planned messaging and notification features.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviations used in the report

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CRM	Customer Relationship Management
DFD	Data-Flow Diagram
DTO	Data Transfer Object
ERD	Entity Relationship Diagram
Expo	Framework and tooling for React Native applications
FYP	Final Year Project
HTTP/HTTPS	Hypertext Transfer Protocol / Secure HTTP
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVP	Minimum Viable Product
OAuth	Open Authorization
OCR	Optical Character Recognition
POS	Point of Sale
REST	Representational State Transfer
SaaS	Software as a Service
SQL	Structured Query Language
UAT	User Acceptance Testing
UI/UX	User Interface / User Experience
UML	Unified Modeling Language
URL	Uniform Resource Locator

CHAPTER 1: INTRODUCTION

1.1 Background

Small enterprises form an important part of Nepal's local economy. Retail shops, wholesalers, service providers and home-based businesses often operate with a small number of employees and limited administrative resources. The owner commonly performs several roles at once: purchasing goods, serving customers, recording credit, following up with leads and reviewing cash flow. Because daily operations take priority, record keeping is frequently based on paper registers, messaging applications or separate spreadsheets.

These tools can support a business at an early stage, but they do not maintain one dependable view of stock, sales and customer activity. A product may be sold without the notebook being updated, a supplier payment may be recorded in another file, or a promising lead may be forgotten after an informal phone conversation. When the owner later asks a simple question—such as which items should be reordered, which customers have unpaid balances, or whether this week performed better than last week—the answer requires manual calculation.

SmartBiz addresses this problem through a mobile-first system supported by a web dashboard. The mobile application is intended for immediate operational tasks at the counter or storeroom, while the web dashboard supports broader review on a larger screen. Both clients use the same backend services and the same authenticated business data. This shared foundation reduces duplicate entry and permits a sale, stock movement or customer update to become visible across the system.

The project was designed as a twelve-week solo final year project under a fixed technology requirement: Java Spring Boot and PostgreSQL must form the backend foundation. The final implementation extends this requirement with service discovery, an API gateway, React Native, Next.js, Redis, Docker Compose and Gemini-assisted analysis. The architecture is intentionally modular so that critical domains can evolve independently without allowing one service to read another service's database directly.

1.2 Problem Statement

Small businesses need an affordable and understandable way to coordinate products, suppliers, sales, customers and leads. Existing manual or fragmented methods create five connected difficulties:

- Inventory counts can become inaccurate because sales and restocking are recorded at different times or in different places.
- Supplier balances, customer due amounts and sales totals require repeated manual reconciliation.
- Customer history and lead follow-up information are not consistently available when a decision must

be made.

- Owners receive little analytical support from raw records and may reorder too late, retain slow-moving stock or miss cash-flow warning signs.
- Using separate applications for each activity increases cost, repeated data entry and learning effort for a small team.

The design problem is therefore not limited to digitising a register. The system must preserve tenant isolation, prevent invalid stock deductions, integrate online payment states, recover when optional infrastructure such as Redis is unavailable, and still present the result through an interface that remains practical on a phone.

1.3 Relevance to Information Technology

SmartBiz applies several core areas of information technology in one working product. The backend demonstrates distributed service design, service discovery, gateway routing, REST communication, database isolation, transactions, caching and containerisation. The clients demonstrate responsive information architecture, secure token storage, asynchronous API integration and mobile device features such as camera input.

The project also explores the responsible placement of generative AI within a business workflow. Gemini is not treated as the system of record. It receives authorised context through the AI service and produces explanations, parsed proposals or insight cards that can be reviewed before a business record is committed. This distinction between deterministic transaction processing and probabilistic assistance is important for reliable software.

Finally, the project is relevant to cybersecurity and data governance. Authentication is performed at the gateway, user identity is propagated to downstream services, and repositories filter by user ID. API responses use DTOs rather than exposing persistence entities. Environment variables separate credentials from source code, and database changes are versioned through Flyway migrations.

1.4 Aim and Objectives

1.4.1 Main Aim

The main aim is to design, implement and evaluate a mobile-first business management system that gives a small business one secure platform for daily operations, business records and decision support.

1.4.2 Specific Objectives

- Implement secure signup, login, email verification, password recovery, Google OAuth, profile management and tenant-aware access.
- Provide product, category, supplier, stock, supplier-ledger and low-stock management.
- Implement point-of-sale recording, sales history, weekly trends and online-payment-aware stock

reservation.

- Provide customer records, purchase totals, due amounts, interactions and a structured lead pipeline with lead-to-customer conversion.
- Develop React Native and Next.js clients connected to the same backend through the API gateway.
- Use AI for contextual questions, insight cards, invoice interpretation, voice-assisted entry and controlled import reconciliation.
- Apply PostgreSQL, Flyway, DTOs, caching, Docker Compose and automated tests to support reliability and maintainability.

1.5 Scope of the Project

1.5.1 Functional Scope

The implemented scope covers six business areas and their supporting infrastructure:

Table 1.5.1: Functional scope of SmartBiz

Area	Included functions
Identity and billing	Signup, login, verification, password reset, Google OAuth, profile, plan status and payment records.
Inventory	Categories, products, images, barcode lookup, low stock, adjustments, restock, suppliers, balances, ledger and stock reservations.
Sales	POS sale creation, history, import, today/weekly/trend analytics, eSewa workflow and payment-state handling.
CRM	Customer CRUD, purchase total, due amount, interactions, lead stages, follow-up and conversion to customer.
AI assistance	Business questions, insight cards, invoice scan, voice parsing, sales-file parsing, staged import analysis and reconciliation.
User interfaces	Expo mobile application and Next.js web dashboard using shared gateway endpoints.

1.5.2 Technical Scope

The technical scope includes Java 21, Spring Boot 3.4.5, Spring Cloud Gateway, Eureka, PostgreSQL, Redis 7, Flyway, Docker Compose, React Native with Expo, TypeScript and Next.js. Managed external services include Neon PostgreSQL, Gemini, Google OAuth and payment-provider interfaces. The repository contains an optional messaging module, but unified inbox behavior is outside the completed core deployment.

1.6 Limitations

- The project is an academic MVP developed by one person in twelve weeks; it has not completed a production security audit or a high-volume load test.
- Internet connectivity is required for the hosted databases, AI requests, OAuth and online payment verification.
- AI responses may be incomplete or inaccurate and therefore remain advisory; users must review

proposed imports and business recommendations.

- Voice recognition requires a native development build for full device speech support; Expo Go uses a text fallback.
- The Stripe path is disabled by default and the eSewa integration uses configurable test/UAT behavior until production credentials and compliance checks are completed.
- Unified messaging, Firebase push notifications and refresh-token rotation remain planned enhancements.
- The report uses a student-ID placeholder because a verified ID was not available in the project materials.

1.7 Target Users

The primary target user is the owner or manager of a small retail, wholesale or service business in Nepal. Secondary users are trusted staff members who record sales, update stock or follow up with customers. The design assumes limited time for training, frequent phone use and a need for clear summaries rather than complex enterprise configuration.

1.8 Organisation of the Report

Chapter 1 introduces the problem, objectives and scope. Chapter 2 reviews representative business tools and identifies the gap addressed by SmartBiz. Chapter 3 describes requirements, methodology, feasibility, risks and system design. Chapter 4 explains implementation technologies and module behavior. Chapter 5 presents testing evidence and evaluation. Chapter 6 concludes the project and proposes future improvements, followed by references and appendices.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

A literature and product review was undertaken to understand how existing systems address inventory, sales, customer relationships and business analytics. The objective was not to reproduce an enterprise platform, but to identify patterns that are useful for a smaller Nepalese business and to recognise where a focused mobile-first system can reduce complexity.

The review considers four categories: spreadsheet-based record keeping, integrated enterprise suites, specialist inventory or point-of-sale products, and customer relationship platforms. Each category solves part of the problem but introduces trade-offs in cost, configuration, connectivity, learning effort or data fragmentation.

2.2 Spreadsheet and Paper-Based Management

Paper registers are inexpensive, familiar and independent of electricity or network availability. They are useful for quick notes, but their information cannot be searched reliably, summarised automatically or shared between devices. Corrections can be difficult to trace and the same figure may be copied into multiple books.

Spreadsheets improve calculation and filtering, yet they still depend on disciplined manual entry. Concurrent edits, inconsistent formulas and uncontrolled copies can create multiple versions of the truth. A spreadsheet does not naturally enforce a stock reservation, authenticate each user, verify payment state or isolate data by business account. SmartBiz therefore retains the simplicity of list-based entry while adding validation, identity and domain rules.

2.3 Representative Existing Platforms

2.3.1 Odoo

Odoo represents an integrated enterprise suite with modules for inventory, sales, accounting and customer relationships. Its modular scope demonstrates the benefit of shared business data, but configuration and process depth can be disproportionate for a small owner who only needs essential workflows. SmartBiz adopts the integrated-domain idea with a smaller, opinionated feature set.

2.3.2 Zoho Inventory

Zoho Inventory represents a cloud inventory and order-management product. It illustrates the value of product tracking, supplier information and reports. A specialised inventory product may still require a separate CRM or locally suitable payment workflow, causing the user to move between systems.

2.3.3 Square Point of Sale

Square POS demonstrates how a mobile-friendly checkout can connect products, payments and receipts. Its strength is transaction speed, but availability, payment support and surrounding business features vary by region. SmartBiz treats POS as one part of a broader inventory, supplier, customer and lead workflow and includes configurable eSewa-oriented behavior for the Nepalese context.

2.3.4 HubSpot CRM

HubSpot CRM demonstrates structured contacts, leads and pipeline stages. Its approach supports follow-up discipline, but a dedicated CRM does not necessarily maintain the shop's physical stock or make an inventory reservation when payment begins. SmartBiz connects lead conversion and customer totals to the operational sales domain through service APIs.

2.4 Identified Gaps

The review identified the following gaps for the target project context:

- A small business may need inventory, POS, customers, leads and simple analytics without the configuration burden of a broad enterprise suite.
- International products may not align with Nepalese payment conventions or an owner's preference for a phone-first workflow.
- Specialised products can fragment data and require repeated entry between inventory, CRM and accounting-like records.
- Raw dashboards explain what happened but may not translate records into understandable reorder, slow-stock, bundle or cash-flow signals.
- Many systems assume a stable connection and mature operational process; an academic MVP must also degrade safely when optional caching or AI services fail.

2.5 Proposed System

SmartBiz combines the minimum set of business domains required for the identified user. Products, suppliers, sales, customers and leads are separate modules in the user interface but share one identity and gateway. The backend separates these domains into services with independently owned data, reducing accidental coupling while keeping integration explicit.

The system adds AI at review points rather than at irreversible transaction boundaries. A scanned invoice or imported file becomes a structured proposal that the user can inspect. Insight cards are derived from business context but do not alter stock. Sales and payments remain deterministic and are protected by reservations and provider verification.

Mobile and web clients are complementary. The mobile application supports quick operational use, camera input and voice assistance. The web dashboard supports paginated tables and wider analytics. This arrangement responds to the target user's need for mobility without preventing office-based review.

Table 2.5.1: Comparative view of representative approaches

Criterion	Paper / spreadsheet	Specialist product	Enterprise suite	SmartBiz
Integrated inventory, sales and CRM	Low	Usually partial	High	High within MVP scope
Mobile-first daily workflow	Manual / limited	Often high	Varies	Core design goal
Nepal-oriented payment path	Manual	Region dependent	Requires configuration	Configurable eSewa flow
Tenant-aware API architecture	Not applicable	Vendor managed	Vendor managed	JWT + X-User-Id
AI-assisted scan/import	No	Varies	Varies	Review-before-commit workflow
Operational complexity	Low initially; grows with data	Medium	High	Moderate and focused
Offline capability	Paper: high	Varies	Varies	Limited in current MVP

2.6 Conceptual Foundations

2.6.1 Microservice Boundaries

A service boundary should correspond to a cohesive business responsibility and should own its persistence model. SmartBiz applies this principle to identity/billing, inventory, CRM, sales and AI. The gateway and discovery server support communication but do not own business records. Direct cross-service database access is prohibited; shared identifiers are exchanged through REST.

2.6.2 Transactional Integrity

PostgreSQL transactions provide all-or-nothing behavior inside one service database. SmartBiz uses this property for stock updates and reservation commits. Since a database transaction cannot safely span independent services without a distributed transaction mechanism, the sale workflow uses explicit states: reserve, verify, commit or release. This is a compensating workflow rather than a hidden cross-database transaction.

2.6.3 Cache-Aside Resilience

Redis reduces repeated work for paginated list endpoints, but it is treated as an optimisation. Cache configuration uses a lenient error handler so a stale or malformed entry produces a warning and the service falls through to PostgreSQL. Write operations evict affected collections. This preserves correctness when Redis is temporarily unavailable.

2.6.4 Human-in-the-Loop AI

Generative AI is useful for interpreting unstructured input and explaining patterns, but its output is probabilistic. SmartBiz therefore separates an AI proposal from an authoritative

business transaction. Import sessions record state, analysis and reconciliation before commit, providing a review boundary that supports accountability.

2.7 Chapter Summary

The review shows that existing approaches offer valuable individual capabilities but can be too fragmented or too complex for the target user. SmartBiz's contribution is a focused combination of mobile operations, shared data, explicit service boundaries, safe payment-linked stock handling and reviewable AI assistance.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter translates the problem into requirements and describes the design selected to satisfy them. It covers stakeholders, feasibility, methodology, risks, architecture, data ownership, data flow, use cases, the critical sale sequence and interface planning.

3.2 Stakeholder and Requirement Analysis

3.2.1 Stakeholders

Table 3.2.1: Stakeholders and interests

Stakeholder	Primary interest
Business owner	Accurate stock, sales visibility, customer credit, low learning effort and control over access.
Staff user	Fast entry, clear validation and predictable task flow on mobile.
Customer	Accurate sale, payment status and due records.
Supplier	Consistent purchase, payment and adjustment ledger information.
Project evaluator	Evidence that requirements, design, implementation and testing are aligned.
System administrator / developer	Maintainable services, migrations, logs, configuration and recovery behavior.

3.2.2 Functional Requirements

Table 3.2.2: Prioritised functional requirements

ID	Requirement	Priority
FR-01	Register, verify and authenticate a user; recover access and update profile.	Must
FR-02	Create, read, update, search and delete products and categories.	Must
FR-03	Adjust stock, show low-stock products and maintain supplier balances/ledger.	Must
FR-04	Create a sale with validated quantities and maintain sale history.	Must
FR-05	Reserve stock during online checkout and commit only after verified success.	Must
FR-06	Manage customers, due amounts, interactions, leads and lead conversion.	Must
FR-07	Provide weekly/today/trend analytics and dashboard summaries.	Should
FR-08	Answer contextual questions and produce reviewable AI insight cards.	Should
FR-09	Parse invoice images, voice text and sales files into proposals.	Should
FR-10	Support plan/billing state and provider callbacks.	Could

3.2.3 Non-Functional Requirements

Table 3.2.3: Non-functional requirements

Category	Requirement and design response
Security	Signed JWT, gateway validation, tenant header, user-filtered repository queries, validation, DTO responses and externalised secrets.
Reliability	Atomic local transactions, reservation state machine, idempotent payment checks, cache fall-through and explicit error responses.
Performance	Pagination, Redis caching, compact DTOs and independent scaling of service containers.
Usability	Mobile-first navigation, search/filter patterns, status badges, review screens and consistent feedback.
Maintainability	Layered services, domain DTOs, Flyway migrations, TypeScript service modules, shared tokens and Docker configuration.
Portability	Java 21 containers, environment variables and Docker Compose across supported hosts.
Auditability	Persisted sale/payment/import states and append-like supplier/customer interaction records.

3.3 Development Methodology

An incremental methodology was selected because the project combined several dependent domains and had a fixed twelve-week schedule. Work was divided into short vertical slices: infrastructure and identity, inventory, CRM, sales, clients, AI assistance, billing, caching and evaluation. A slice was considered usable only when its endpoint, persistence, client integration and error handling could be exercised together.

This approach reduced the risk of producing isolated screens or endpoints. For example, the inventory feature was not treated as complete when product CRUD existed; it was extended through mobile/web integration, pagination, supplier behavior and stock reservation. Git provided version history, while Docker Compose provided a repeatable integration environment for all services.

Table 3.3.1: Incremental project phases

Phase	Major outputs
Foundation	Requirements, architecture, repository structure, Eureka, gateway and environment strategy.
Core domains	Authentication, inventory/suppliers, CRM/leads and database migrations.
Transactions	Sales, stock checks, reservation/commit/release and analytics.
Experience	Expo mobile screens, Next.js dashboard, pagination and reusable UI patterns.
Intelligence and billing	Gemini workflows, import sessions, insight cards, OAuth and payment providers.
Quality and delivery	Caching, exception handling, tests, Docker builds, documentation and evaluation.

3.4 Feasibility Study

3.4.1 Technical Feasibility

The chosen technologies are compatible with the project constraint and available development environment. Java 21 supports the Spring Boot services, PostgreSQL is available locally or through Neon, and Expo supports development on Windows with Android devices. Docker Compose can start the distributed backend from one configuration. The principal technical risks were service integration, payment state and native speech support; each received a fallback or explicit state model.

3.4.2 Operational Feasibility

The workflows mirror familiar small-business activities: add product, record sale, update customer and review totals. Search, filters, status badges and quick actions reduce the number of steps. The mobile application does not require a user to understand the underlying microservices. Operational adoption still depends on initial product entry and consistent use at the point of sale.

3.4.3 Economic Feasibility

The project uses open-source frameworks and can run in low-cost containers. Managed PostgreSQL, AI and payment services may introduce usage-based costs, but the architecture permits local development and configurable integrations. For an academic MVP, the primary cost was development time rather than software licensing.

3.4.4 Schedule Feasibility

The scope was organised around a twelve-week plan. Core inventory, sales, CRM and authentication were prioritised before optional messaging and push notifications. This prioritisation enabled a complete business workflow while moving lower-priority features to the future roadmap.

3.5 Data Collection and Analysis

Requirements were derived from observation of common retail record-keeping tasks, informal discussions about stock and credit tracking, inspection of representative software categories and continuous review of the working prototype. The analysis focused on repeated pain points rather than attempting a statistically generalisable market study. Therefore, conclusions are appropriate for project design but should be validated with structured field research before commercial deployment.

3.6 SWOT and Risk Analysis

Table 3.6.1: SWOT analysis

Strengths	Weaknesses	Opportunities	Threats
Integrated mobile/web workflow; modular services; tenant isolation; reviewable AI; payment-aware stock reservation.	Internet dependency; solo-project test capacity; operational complexity of microservices; limited offline support.	Nepal-focused payments; notifications; messaging; richer forecasting; multi-branch support; SaaS deployment.	Credential leakage; provider changes; AI rate limits; user resistance; data quality; network outages.

Table 3.6.2: Principal risks and mitigations

Risk	Likelihood / impact	Mitigation
Cross-user data exposure	Low / Critical	Gateway identity propagation plus user-scoped repository queries and endpoint tests.
Stock deducted for failed payment	Medium / High	Reserve first; verify provider; commit on success; release on failure/expiry.
Redis outage or malformed entry	Medium / Medium	Lenient cache error handler and database fall-through.
AI hallucination or parse error	Medium / High	Review-before-commit import sessions and deterministic validation.
External API rate limit	Medium / Medium	Clear errors, configurable Gemini model/key and non-AI core workflows.
Schedule expansion	High / Medium	Prioritised MVP; defer messaging, push and refresh-token work.

3.7 System Architecture

Both user interfaces send HTTPS/JSON requests to the API gateway. The gateway validates JWTs, derives the authenticated user and routes requests to services discovered through Eureka. Each domain service owns a PostgreSQL database. Redis caches selected paginated reads, and the AI service is the only component that calls Gemini. This arrangement is shown in Figure 3.7.1.

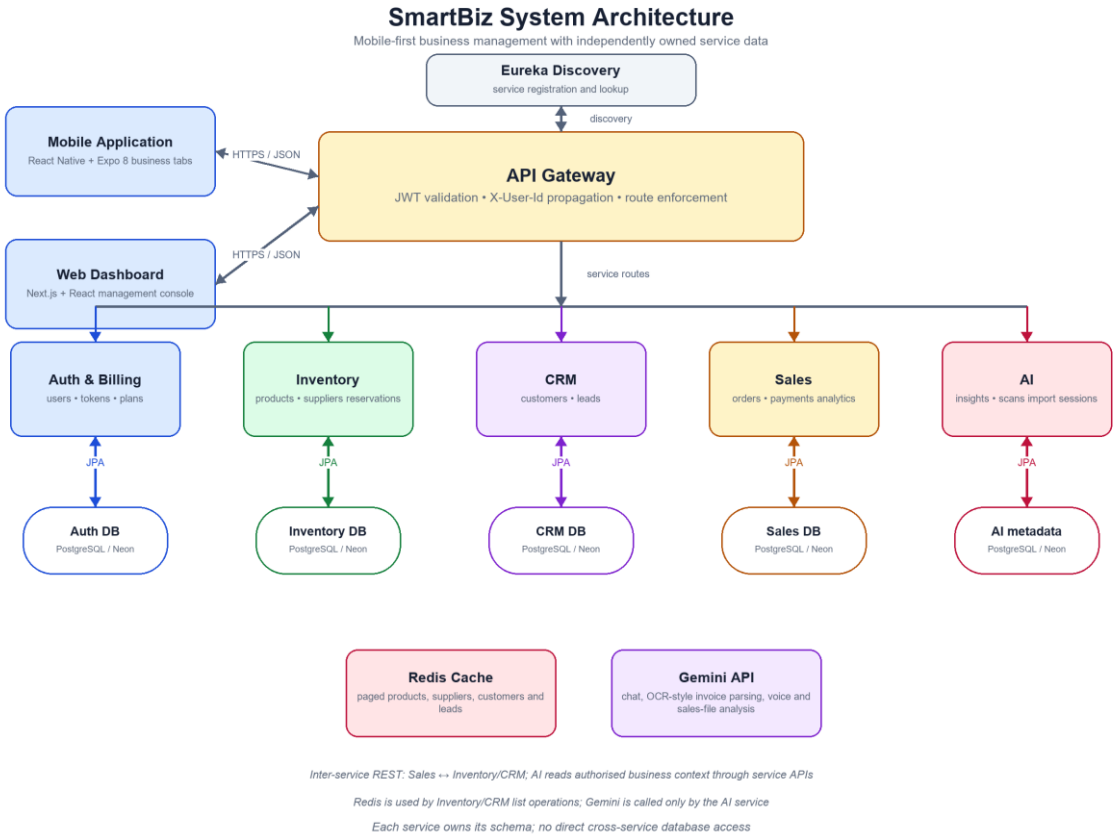

Figure 3.7.1: SmartBiz system architecture

The architecture deliberately separates availability concerns. A cache failure should not become a data failure, and an AI failure should not prevent product or sale management. The gateway provides one external entry point while the internal network retains named service endpoints. Shared payment DTOs and signatures are placed in a small common library without sharing domain persistence.

3.8 Data Design

The database-per-service strategy means there is no single global schema. Foreign keys are used only within the database that owns both entities. A sales record can carry a customer ID or product ID received through an API, but the sales database cannot enforce a foreign key into the CRM or inventory database. Service logic validates and reconciles these references.

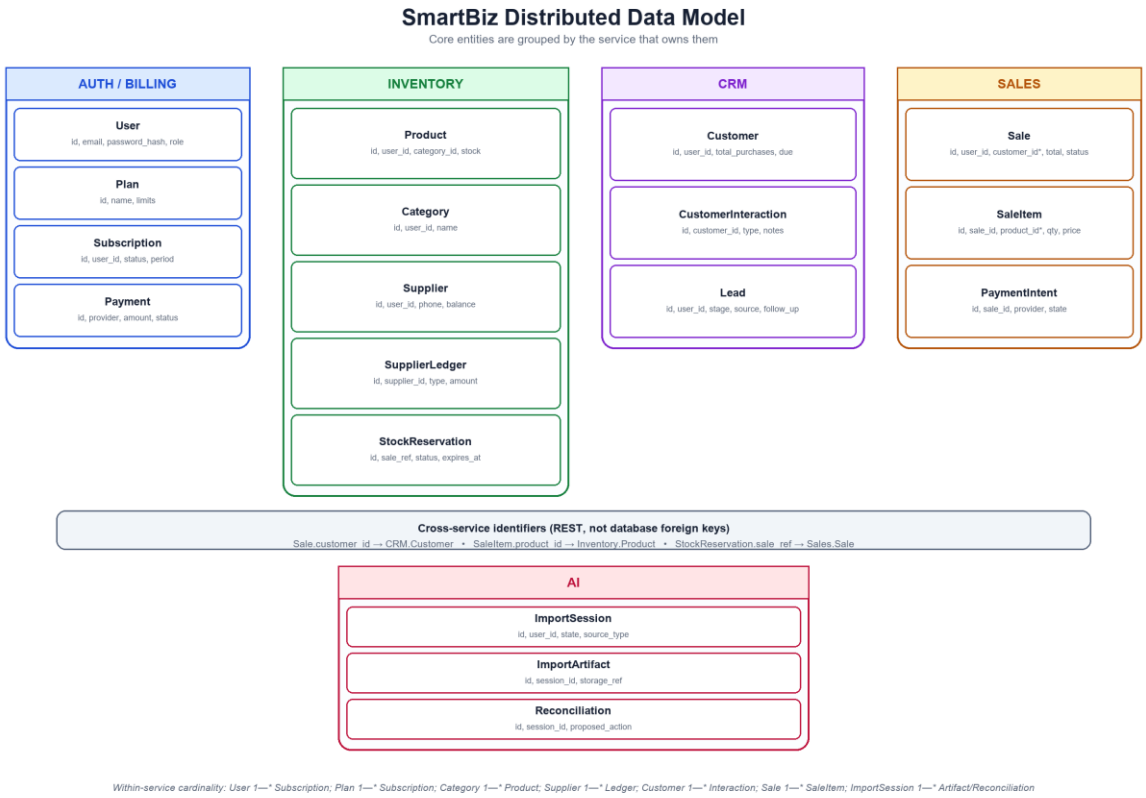

Figure 3.8.1: Distributed data model and service ownership

Table 3.8.1: Data ownership by service

Service	Owned records	Selected integrity rules
Auth/Billing	Users, verification/reset tokens, plans, subscriptions and provider payments.	Unique identity, token expiry and controlled subscription state.
Inventory	Categories, products, suppliers, ledger, images and stock reservations.	Non-negative quantities, user ownership and reservation status.
CRM	Customers, interactions and leads.	Tenant filtering, lead-stage values and conversion rules.
Sales	Sales, sale items, analytics inputs and payment intents.	Calculated totals, sale status and provider verification state.
AI	Import sessions, artifacts and reconciliation metadata.	State transitions and user-approved commit boundary.

3.9 Data-Flow Design

3.9.1 Context Diagram

At context level, the business owner interacts with SmartBiz as one system. Customers provide sale/payment information, while external providers supply identity, payment and AI responses.

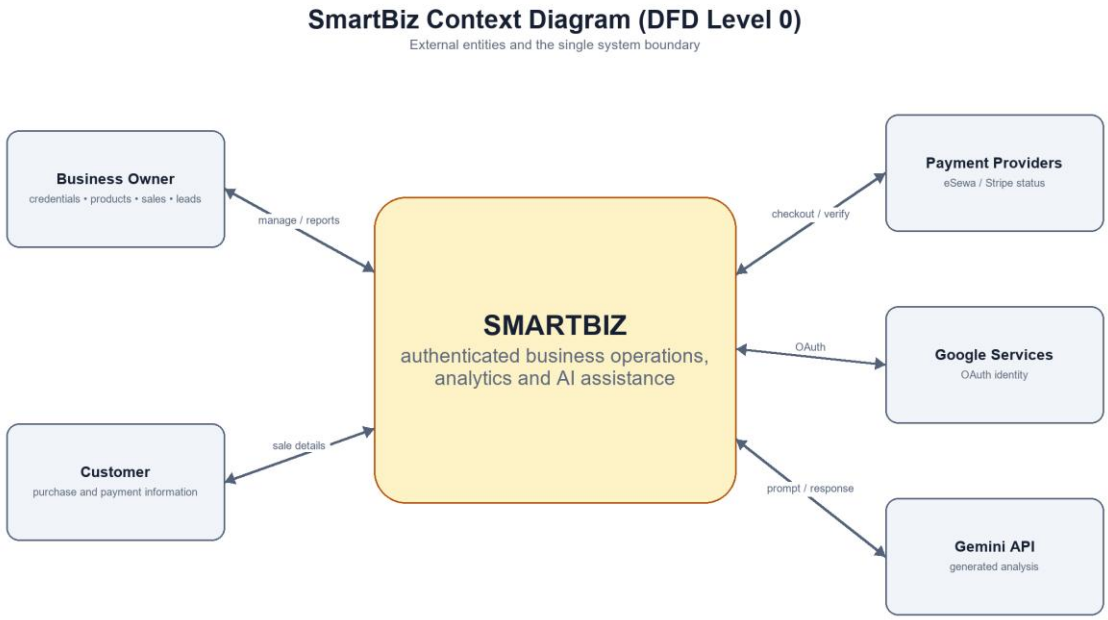

Figure 3.9.1: SmartBiz context diagram (DFD Level 0)

3.9.2 Level-1 Data Flow

The level-1 view separates identity, inventory, CRM, sales, AI and import processes. Data stores are independently owned and external providers are reached through controlled service calls.

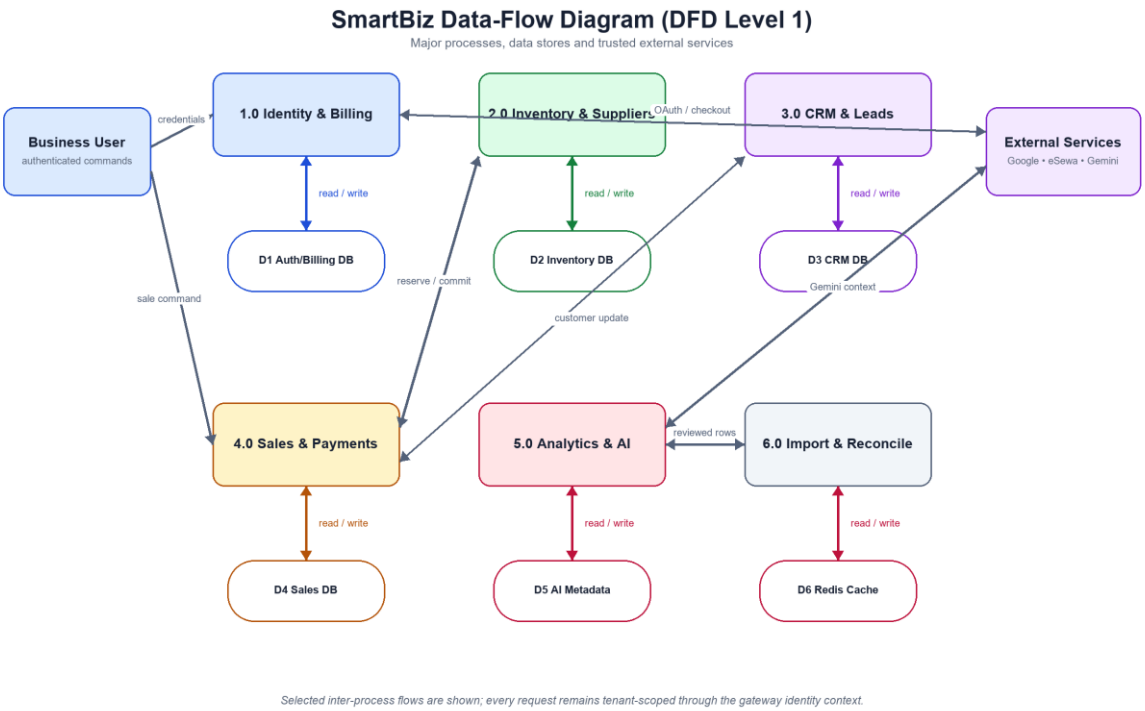

Figure 3.9.2: SmartBiz data-flow diagram (DFD Level 1)

3.10 Use-Case Design

The business owner can access all operational and administrative features. A staff user performs routine stock, sales and customer activities subject to the account's authorization rules. Payment providers and Gemini participate only in bounded use cases.

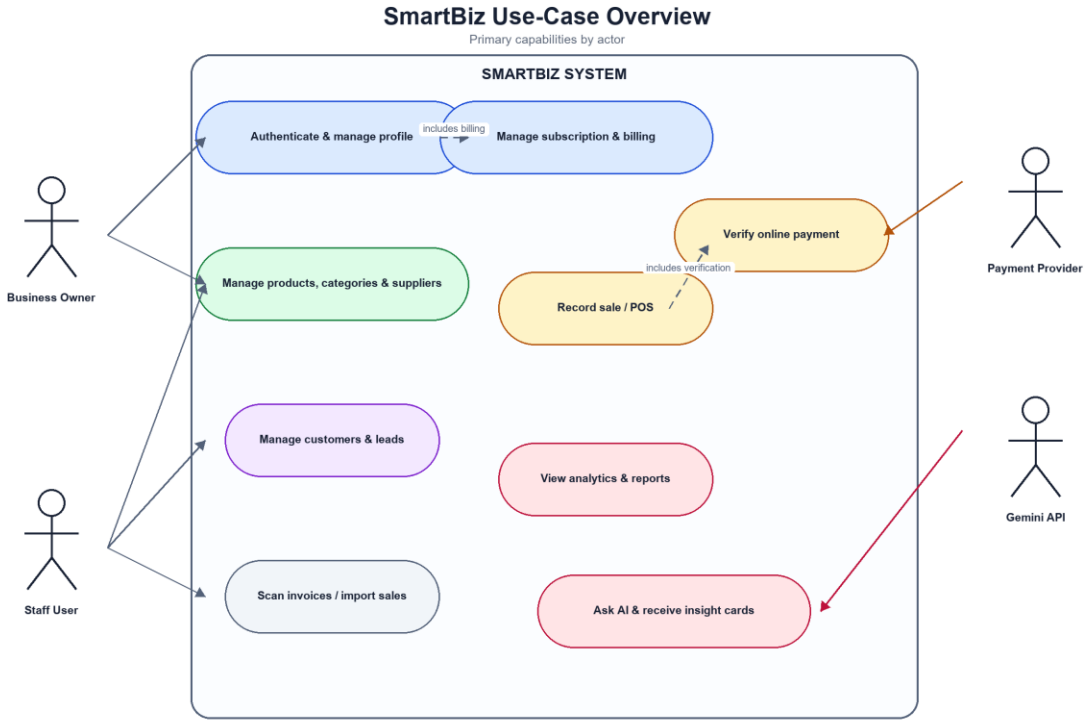

Figure 3.10.1: SmartBiz use-case overview

3.11 Critical Sale and Payment Sequence

A direct cash sale can validate stock and commit the sale without an external checkout. An online payment requires a longer sequence. The sales service first requests an inventory reservation. A provider intent is then created and the customer completes the external flow. The callback alone is not trusted; the sales service verifies provider status before committing the reservation.

If the provider reports failure, cancellation or expiry, the reservation is released. The stock deduction is atomic inside the inventory database, but the complete cross-service workflow is not described as one distributed database transaction. Its safety comes from explicit state transitions, verification and compensation.

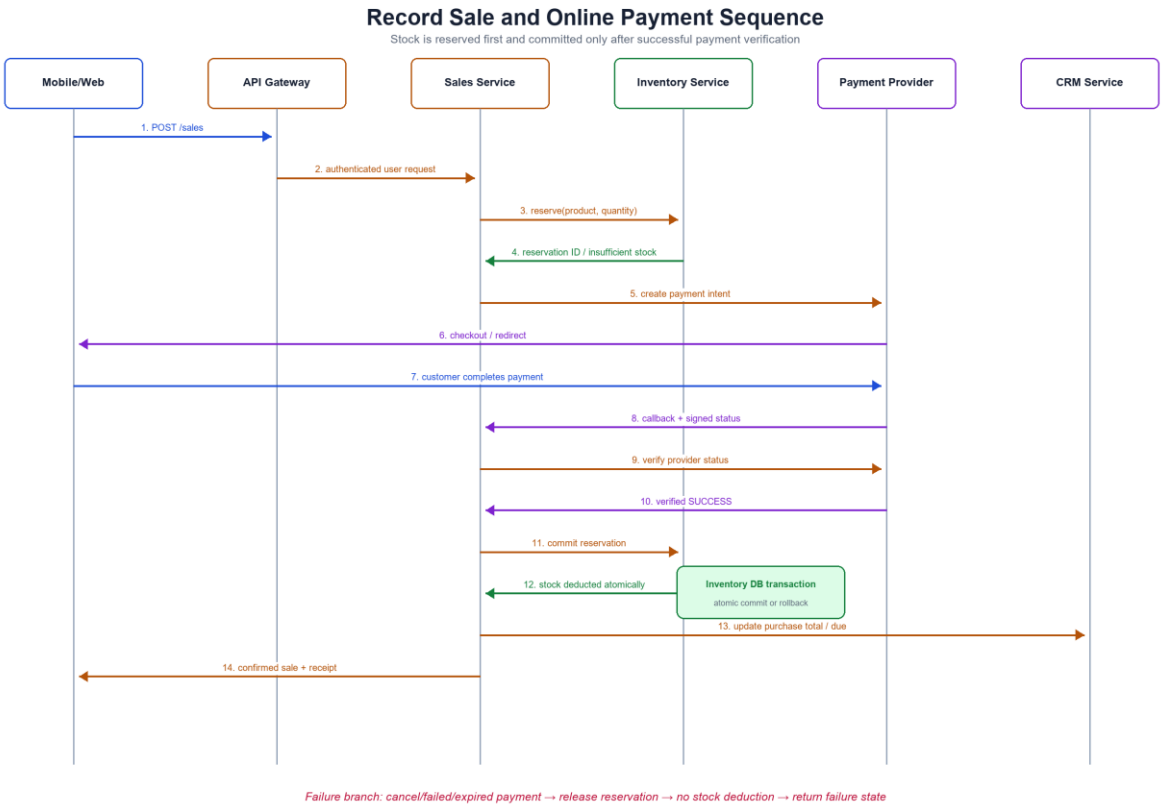

Figure 3.11.1: Record-sale and online-payment sequence

3.12 Deployment Design

The backend is packaged as containers on an internal Compose network. Mobile and web clients access the public gateway, while PostgreSQL, Gemini and payment providers remain external managed services. Health checks and environment variables support repeatable startup and configuration.

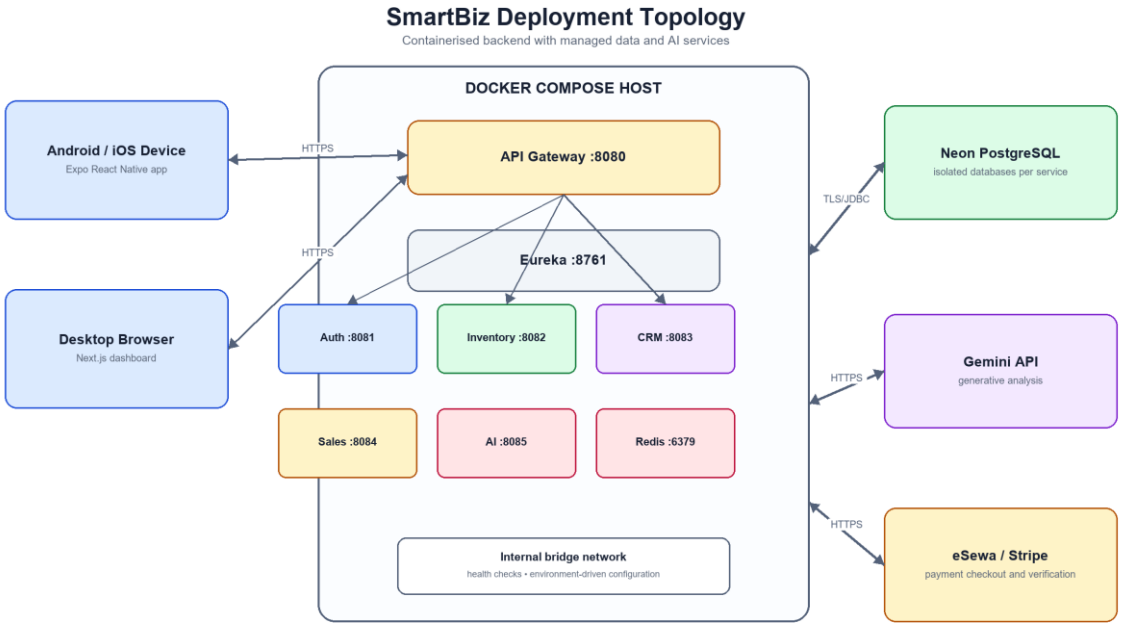

Figure 3.12.1: SmartBiz deployment topology

3.13 Interface Design

The interface uses repeated patterns to reduce learning effort: search at the top, filters near the result list, status badges for state, expandable detail cards on mobile and paginated tables on web. Primary actions are visually prominent, while destructive actions require deliberate selection.

The mobile navigation exposes the most frequently used business domains as tabs. Camera and voice actions appear in context instead of as separate applications. The web dashboard preserves the same concepts but uses wider tables, previous/next pagination and larger analytics surfaces.

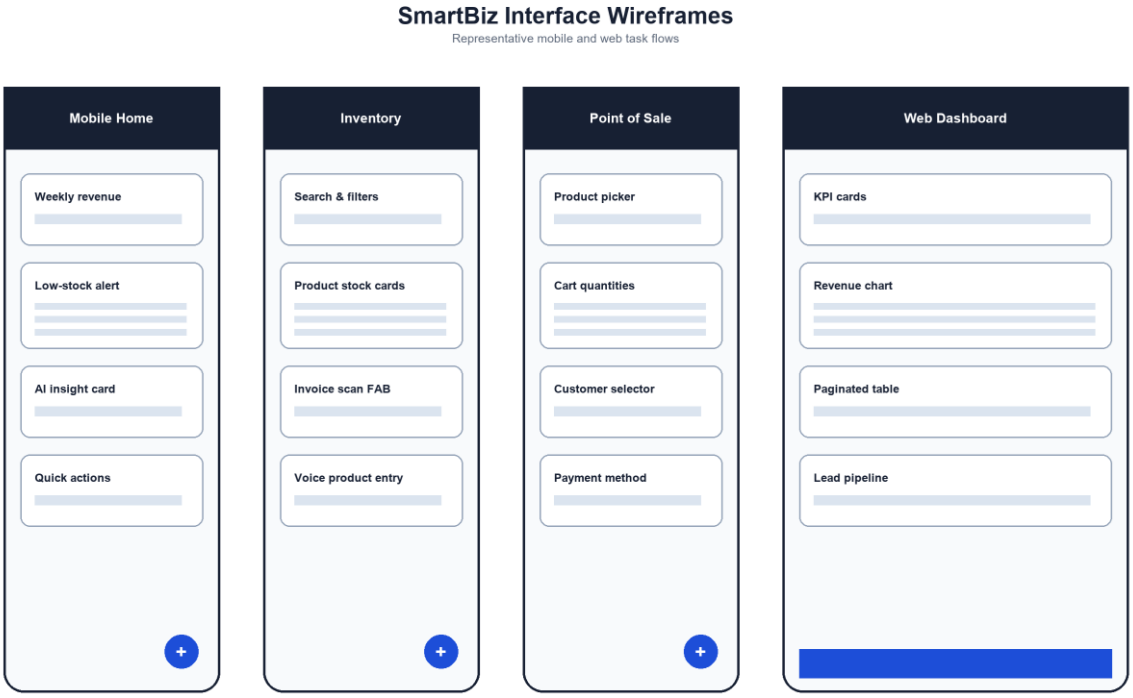

Figure 3.13.1: Representative mobile and web wireframes

3.14 Chapter Summary

The design connects usability requirements to explicit technical controls. The next chapter explains how these designs were implemented in the repository.

CHAPTER 4: IMPLEMENTATION

4.1 Introduction

Implementation followed the service boundaries and incremental plan established in Chapter 3. This chapter describes technologies, repository organisation, backend modules, client applications, security, caching, AI integration, deployment and project resources.

4.2 Technologies and Tools

Table 4.2.1: Main implementation technologies

Layer	Technology	Purpose in SmartBiz
Backend language/runtime	Java 21	Required runtime for all Spring services and shared payment library.
Backend framework	Spring Boot 3.4.5	REST controllers, validation, transactions, security, data access and configuration.
Gateway/discovery	Spring Cloud Gateway and Eureka	Single entry point, routing, JWT identity propagation and service lookup.
Database	PostgreSQL 15+ / Neon	Relational persistence with transactions and per-service databases.
Migration	Flyway	Versioned schema creation and forward-only changes.
Cache	Redis 7	Cached paginated reads with database fall-through on errors.
Mobile	React Native 0.81.5, Expo 54, TypeScript	Phone-first operational interface and device features.
Web	Next.js 16.2.4, React 19.2.4, Tailwind CSS 4	Server/client dashboard, tables and analytics interface.
AI	Gemini API	Contextual chat, insight cards, scan/voice/file interpretation.
Deployment	Docker and Docker Compose	Multi-stage images, internal networking and repeatable startup.

4.3 Repository Organisation

```
smartBiz/
  backend/
    eureka-server/                                              api-gateway/
    auth-service/                                             inventory-service/
    crm-service/                                                  sales-service/
    ai-service/                                                 payment-common/
  mobile/                                        Expo  /  React  Native  application
  frontend/web/                                            Next.js    dashboard
  docker-compose.yml      project documentation
```

Within each Spring service, controllers expose DTO-based APIs, services implement business rules, repositories isolate persistence, configuration classes define security/caching behavior and Flyway scripts version the schema. Mobile and web clients each use a service layer so UI components do not construct authentication headers independently.

4.4 Backend Infrastructure

4.4.1 Eureka Server

Eureka runs on port 8761 and provides registration and lookup for named services. This allows gateway routes to target logical service names instead of fixed container IP addresses. Health and registration behavior are part of the Compose startup sequence.

4.4.2 API Gateway

The gateway runs on port 8080 and routes /auth and /billing to Auth, /inventory to Inventory, /customers and /leads to CRM, /sales to Sales, and /ai to the AI service. Its authentication filter validates the bearer token, extracts the user identity and supplies X-User-Id to downstream services. Public authentication and provider-callback routes are handled explicitly.

4.5 Auth and Billing Service

The Auth service stores users and account-related state. Registration validates the submitted identity and password, while login returns the token used by both clients. Email verification, resend, forgot-password and reset-password flows use time-limited tokens. Google OAuth provides an alternative identity path. Profile updates return DTOs rather than the user entity.

Billing endpoints expose plan, status, checkout and payment history behavior. Provider-specific details are kept behind service interfaces. eSewa and Stripe configurations are environment-driven and may remain disabled or in test mode when credentials are not supplied. Callback handlers validate provider information before changing payment or subscription state.

4.6 Inventory and Supplier Service

Inventory provides paginated product and supplier endpoints as well as category management. Product records include user ownership, category, stock, price, barcode and optional image information. Search and filters are applied within the user's data set. Signed image operations prevent an upload from becoming a permanent product attachment until the update succeeds.

Supplier management includes summary values, payment and adjustment operations and a ledger. This makes supplier balance changes traceable instead of storing only an unexplained total. Stock endpoints support manual adjustment, restock and low-stock queries.

Reservations are central to safe online sales. A reservation temporarily holds the requested quantity against a sale reference and has an expiry/status. Commit performs an atomic inventory update; release returns the held quantity to availability. Internal endpoints require trusted service context rather than being exposed as normal user operations.

4.7 CRM Service

CRM owns customers, interactions and leads. Customer list endpoints are paginated and tenant-filtered. Purchase totals and due amounts can be updated through controlled endpoints used by the sales workflow. Interactions record the type and notes for later review.

Leads progress through NEW, CONTACTED, INTERESTED, PROPOSAL, WON or LOST. Source, estimated value, notes and follow-up date support basic pipeline management. Conversion creates a customer from a qualified lead and removes the lead inside one CRM transaction. The conversion does not require another service database.

4.8 Sales Service

Sales receives a user-scoped sale request, validates its items and coordinates required inventory and CRM calls. It stores the sale and item values needed for history and analytics. Today, weekly and trend endpoints aggregate authoritative sale data rather than relying on client-side totals.

For online eSewa flow, the service creates an intent only after inventory can be reserved. Provider callbacks are followed by status verification. A verified success commits stock and confirms the sale; failure, cancellation or expiry releases the reservation. Retry and state checks reduce duplicate effects. Customer totals or due amounts are updated through CRM APIs after the sale state is known.

File-imported sales enter through validated import endpoints. The client or AI service may propose rows, but the sales service remains responsible for final domain validation and persistence.

4.9 AI Service

The AI service obtains authorised inventory, sales and CRM context through service APIs and constructs a bounded prompt for Gemini. It supports general business questions and structured insight cards such as slow-moving stock, bundle opportunities, cash-flow warnings and restock signals. The service does not query another service’s database.

Invoice and voice endpoints transform unstructured input into proposed products, leads or chat text. Sales-file import uses sessions, artifacts, analysis, reconciliation, commit and close steps. This staged design gives the user an opportunity to review ambiguous mappings before records are written. Gemini 2.5 Flash Lite is configurable because earlier model availability and regional quota behavior can vary.

4.10 Mobile Application

The Expo application uses TypeScript and file-based routing. SecureStore retains

authentication data, and a shared Axios interceptor attaches Authorization and X-User-Id values. The principal tabs cover home, inventory, suppliers, sales, customers, leads, AI and settings. Product entry, POS cart behavior, lead stages, customer history, analytics and account changes all call the gateway. Camera, image picker and speech-recognition integrations are wrapped with review or fallback behavior.

Table 4.10.1: Mobile navigation and responsibilities

Tab	Principal responsibilities
Home	Weekly revenue, alerts, quick actions and on-demand AI insight.
Inventory	Products, search, stock actions, invoice scan and voice-assisted product entry.
Suppliers	Supplier details, balance status, ledger and adjustments.
Sales	POS cart, customer/payment selection, receipt state and history.
Customers	Customer cards, due values and interaction/history review.
Leads	Stage filters, details, follow-up and conversion.
AI	Contextual chat plus camera, voice and import entry points.
Settings	Profile and account preferences.

4.11 Web Dashboard

The Next.js dashboard implements the same major domains for a larger screen. Server pages read pagination values from asynchronous search parameters and request fifteen-row pages. A shared Pagination component renders Previous and Next links and hides when only one page exists. Inventory, suppliers, customers and leads use paginated tables; sales, analytics, AI and billing pages use the same authenticated service layer.

4.12 Security and Tenant Isolation

Security is enforced as a chain rather than a single check:

- The Auth service issues signed, expiring JWTs after successful identity validation.
- The gateway rejects invalid protected requests and derives X-User-Id from the verified token.
- Each service receives user identity and repository methods include the user ID in lookups and list queries.
- DTOs restrict response fields and request validation rejects structurally invalid input.
- Internal service endpoints use trusted tokens/headers and are not presented as public user APIs.
- Secrets, database URLs and provider credentials are supplied through environment variables and are not printed in this report.
- Provider callbacks and payment status are verified before stock or subscription effects are committed.

A remaining production improvement is refresh-token rotation with revocation and tighter issuer/audience enforcement across all services. HTTPS termination, secure cookie decisions

for web OAuth, rate limiting and an external security review are also part of the hardening roadmap.

4.13 Pagination and Redis Caching

Large list endpoints return a PagedResponse containing content, current page, total pages, total elements and hasNext. This contract is used by both mobile load-more behavior and web previous/next navigation. Page and size are included in cache keys so one tenant or page cannot return another tenant's result.

Write methods evict affected caches. Both inventory and CRM cache configurations implement a lenient error handler: read/write/evict failures are logged as warnings and the request continues to the database. Redis therefore improves response efficiency without becoming a required source of truth.

4.14 API Design and Error Handling

Endpoints use resource-oriented paths and JSON DTOs. Common operations map to GET, POST, PUT and DELETE, while domain actions such as /convert, /reserve, /commit or /release use explicit POST endpoints. Global exception handlers translate not-found, validation, conflict and internal failures into appropriate status codes and concise response bodies. Clients display recoverable messages rather than exposing stack traces.

Table 4.14.1: Representative API routes

Method and route	Service	Purpose
POST /auth/login	Auth	Authenticate and issue access token.
GET /inventory/products?page=&size=	Inventory	Return tenant-scoped product page.
POST /inventory/reservations	Inventory	Reserve stock for a sale reference.
POST /leads/{id}/convert	CRM	Create a customer from a qualified lead.
POST /sales	Sales	Validate and record a sale.
POST /sales/payments/esewa	Sales	Create payment intent after reservation.
GET /sales/analytics/weekly	Sales	Return weekly aggregate values.
POST /ai/import-sessions	AI	Begin reviewable sales import.

4.15 Deployment and Configuration

Docker Compose builds and starts Eureka, the gateway, core services and Redis on a shared network. Multi-stage images reduce runtime contents. Environment variables provide database URLs, credentials, JWT secret, Eureka URL, Gemini key, OAuth values and provider configuration. Spring configuration includes safe local fallbacks where appropriate, while

production secrets must be supplied by the deployment platform.

4.16 Schedule and Resource Use

Figure 4.16.1: Twelve-week SmartBiz development plan

Table 4.16.1: Project resources

Resource type	Items used
Hardware	Windows development computer, Android test device and internet connection.
Development software	JDK 21, Maven, Node.js/npm, code editor, Git, Docker Desktop and PostgreSQL tooling.
Frameworks	Spring Boot/Cloud, React Native/Expo, Next.js, Redis and Flyway.
Hosted services	Neon PostgreSQL, Gemini API, Google OAuth and payment-provider test environments.
Documentation	Project context, change log, source comments, framework documentation and this report.

4.17 Chapter Summary

The implementation realises the planned domain boundaries and connects mobile and web clients through one authenticated gateway. It also introduces resilience patterns—pagination, cache fall-through, reservations and reviewable AI—that are directly tied to the requirements.

CHAPTER 5: TESTING AND EVALUATION

5.1 Introduction

Testing evaluated individual service rules, client-facing workflows, cross-service integration assumptions and deployment behavior. This chapter reports observed evidence without inventing performance figures that were not measured.

5.2 Testing Strategy

The strategy combined the following levels:

- Unit tests for payment signatures, controllers and service business rules.
- Repository/service tests for user isolation, pagination, reservations and conversion behavior.
- Integration-oriented tests using mocked service/provider boundaries for sale and payment state.
- Manual workflow checks through the mobile and web clients against gateway endpoints.
- Container startup and service-registration checks through Docker Compose.
- Documented negative cases for invalid input, insufficient stock, failed payment, cache failure and AI/provider failure.

5.3 Automated Test Execution

Tests were executed with the project's required Java 21 runtime. The repository contained 58 discovered @Test methods across the shared payment library and five core business services. Fifty-seven passed and one failed. An initial attempt under Java 25 produced a Mockito/Byte Buddy compatibility failure; this was an environment mismatch rather than an application result, and the suite was rerun correctly under Java 21.

The remaining failure occurs in the AI insight-card test. For its fixture, the service returned SLOW_MOVING_STOCK, BUNDLE_OPPORTUNITY and CASH_FLOW_WARNING, while the test also expected RESTOCK_SOON. This is recorded as an unresolved inconsistency between the fixture/expectation and current restock-selection logic. The failed assertion should be resolved before claiming a completely green build.

Table 5.3.1: Automated test result by module

Module	Discovered	Passed	Failed	Observation
payment-common	1	1	0	Payment signature behavior passed.
auth-service	12	12	0	Authentication, billing and controller rules passed.
inventory-service	21	21	0	Product, supplier, stock and reservation cases passed.
crm-service	2	2	0	Customer/lead service cases passed.
sales-service	19	19	0	Sale, payment and integration-oriented rules passed.

Module	Discovered	Passed	Failed	Observation
ai-service	3	2	1	Restock insight-card expectation remains unresolved.
Total	58	57	1	98.3% of discovered tests passed.

5.4 Functional Test Cases

Table 5.4.1: Representative functional tests

ID	Scenario	Expected result	Result
TC-01	Valid login	JWT and user profile are returned.	Pass
TC-02	Request another user's product	No record is returned or access is rejected.	Pass
TC-03	Create product and restock	Product persists and stock increases by validated amount.	Pass
TC-04	Sale with insufficient stock	Sale is rejected; stock remains unchanged.	Pass
TC-05	Successful online payment	Reservation is verified and committed; sale confirms.	Pass
TC-06	Cancelled/failed payment	Reservation is released; stock is not deducted.	Pass
TC-07	Convert lead to customer	Customer is created and lead removed in CRM transaction.	Pass
TC-08	Malformed Redis entry	Warning is logged and database response is used.	Pass by design/test coverage
TC-09	AI import proposal	Rows remain reviewable until explicit commit.	Pass
TC-10	Insight-card restock fixture	Expected card set includes RESTOCK_SOON.	Fail—known issue

5.5 Security and Isolation Evaluation

The strongest security property demonstrated by the design is consistent tenant scoping. The gateway derives user identity from the token rather than accepting an arbitrary client-supplied tenant as authoritative. Downstream queries include the user ID, and cross-service calls propagate trusted identity context.

The assessment also identified areas for production hardening. Shared-secret JWT validation requires careful key management; issuer, audience and expiration claims should be enforced consistently. Refresh-token rotation and revocation are not implemented. Rate limiting, structured security logs, dependency scanning, external penetration testing and production TLS/header configuration should be completed before real financial or personally identifiable data is onboarded.

5.6 Reliability and Failure Evaluation

Table 5.6.1: Failure behavior evaluation

Failure	Designed behavior	Evaluation
Insufficient stock	Reservation or sale request fails before commit.	Prevents negative stock in normal workflow.
Provider cancellation/failure	Release reservation and keep sale unconfirmed.	Separates checkout attempt from inventory effect.
Duplicate callback/retry	Check persisted state before applying effect again.	Reduces duplicate commit risk.
Redis unavailable/corrupt	Log warning and query PostgreSQL.	Cache does not become a single point of failure.
Gemini rate limit/error	Return controlled error; core CRUD/POS remains usable.	AI is optional to system-of-record tasks.
Downstream service unavailable	Return explicit failure; do not silently fabricate success.	Requires retry/observability improvements for production.

5.7 Performance Evaluation

Formal concurrent load testing was not completed, so the report does not claim a measured response-time or throughput target. Performance readiness was instead evaluated through structural controls: paginated endpoints bound response size, Redis avoids repeated list work, database indexes accompany migration design, and service containers can be monitored or scaled separately.

The next evaluation stage should establish a reproducible data set and run gateway-level tests for p50, p95 and p99 latency, error rate, throughput and resource use. Tests should cover cold/warm cache behavior, a Redis outage, concurrent reservations for the final stock unit, payment callback bursts and AI requests under provider limits.

5.8 Objective Achievement

Table 5.8.1: Evaluation against objectives

Objective	Evidence	Status
Secure tenant-aware identity	Auth flows, gateway validation and user-scoped repositories.	Achieved for MVP
Inventory and supplier control	CRUD, search, low stock, ledger, images and reservations.	Achieved
Sales and analytics	POS, payment states, history and aggregate endpoints.	Achieved
CRM and lead pipeline	Customers, due, interactions, stages and conversion.	Achieved
Mobile and web access	Expo mobile tabs and Next.js dashboard use shared APIs.	Achieved
AI-assisted operations	Chat, cards, invoice/voice/file parsing and staged import.	Achieved with known test issue
Maintainable deployment	Flyway, DTOs, caching, Docker Compose and tests.	Achieved for academic MVP

5.9 Challenges Encountered

- Coordinating stock, sale, customer and payment state without sharing databases required an explicit reservation workflow.
- Gateway routes and service security matchers had to remain aligned when adding new resource paths.
- Spring Boot does not automatically load a root .env file during local Maven execution, while Docker Compose does; configuration needed clear instructions and fallbacks.
- Gemini regional model availability and rate limits required a configurable model and graceful failure handling.
- Native speech recognition is unavailable in Expo Go, requiring a development build and a text fallback.
- Stale or malformed cache entries initially risked repeated failures; lenient cache error handlers were introduced.
- Test execution under the wrong Java version produced tooling incompatibility; the authoritative run used Java 21.

5.10 Discussion

The evaluation indicates that SmartBiz meets its main academic objective: it provides one coherent workflow across the principal records of a small business. The strongest aspect is not the number of screens, but the alignment between UI actions and backend controls. A payment attempt does not automatically become a stock deduction, a cache entry is not treated as authoritative data, and an AI interpretation is not committed without review.

The microservice design demonstrates relevant distributed-system concepts, although it carries more operational complexity than a monolith would for a single small business. For a commercial product, the separation would be justified by SaaS growth, independent scaling and team ownership. For the academic context, it provides a valuable implementation of boundaries and inter-service consistency, but deployment and observability require further maturity.

The known AI test failure prevents a claim of perfect automated verification. Its documentation is itself important: quality reporting should reflect actual evidence. The issue is bounded to the composition of advisory insight cards and does not indicate a failed stock or payment transaction. Nevertheless, it should be investigated and the full build rerun before release.

5.11 Chapter Summary

Testing confirms broad functional coverage and validates the most critical inventory, sales, authentication and CRM rules. The outstanding AI assertion, absent load benchmark and production-hardening items define a clear next quality stage.

CHAPTER 6: CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

SmartBiz was developed to replace fragmented small-business records with one mobile-first system supported by a web dashboard. The completed MVP integrates identity, billing, inventory, suppliers, sales, payments, analytics, customers, leads and AI-assisted workflows through a Spring Boot and PostgreSQL architecture.

The project demonstrates that reliable business behavior depends on the relationships between modules. Tenant identity is carried from the gateway into every query. Stock is reserved before online checkout and committed only after verification. Customer totals are updated through CRM APIs rather than shared tables. Redis improves list performance but can fail without blocking database access. Gemini interprets and explains information but is separated from authoritative transaction rules.

The mobile application makes routine operations available at the point of work, while the web dashboard supports wider review. Flyway migrations, DTOs, containers, pagination and automated tests improve maintainability. The observed result—57 of 58 tests passing under Java 21—shows strong coverage for an academic MVP while transparently identifying one remaining AI insight-card inconsistency.

Overall, SmartBiz achieves its central aim and provides a credible foundation for further research and product development. It is not yet a production financial platform, but its architecture and completed workflows demonstrate how a focused system can make small-business information more consistent, visible and actionable.

6.2 Future Enhancements

6.2.1 Immediate Quality Improvements

- Resolve the RESTOCK_SOON insight-card fixture/logic mismatch and restore a fully green test run.
- Add gateway-level integration tests and concurrent reservation tests with real PostgreSQL and Redis containers.
- Introduce structured logs, correlation IDs, metrics, traces, dashboards and alert thresholds.
- Complete a security review covering JWT claims, secret rotation, OAuth redirect/cookie settings, provider signatures and dependency vulnerabilities.
- Run repeatable load tests and publish latency, throughput, error and resource-use results.

6.2.2 Product Enhancements

- Unified inbox and business messaging, supported by the optional messaging service.
- Firebase push notifications for low stock, follow-up dates, payment state and expiring reservations.
- Refresh-token rotation, device/session management and user-initiated revocation.
- Offline-first mobile capture with conflict-aware synchronisation for unreliable connectivity.

- Multi-branch stock, transfer orders, role permissions, purchase orders and richer supplier analytics.
- Tax-aware receipts, Nepali calendar/localisation options and production payment-provider certification.
- Forecast evaluation using explainable historical models and explicit confidence ranges.
- Audit exports, backup/restore processes and configurable data-retention policies.

6.2.3 Research and Evaluation Enhancements

Future study should involve a structured pilot with Nepalese small-business owners. Tasks should measure completion time, error rate, usability, perceived trust in AI suggestions and the effect of alerts on stock availability. Comparative evaluation against spreadsheets or an existing POS would provide stronger evidence of business value than developer testing alone.

6.3 Final Reflection

The project combined software engineering, distributed data management, mobile and web design, security, testing and AI integration within a constrained academic schedule. The most important lesson was to make system states explicit. When identity, payment, reservation, cache and AI-review states are visible in the design, failure behavior can be reasoned about and tested rather than assumed.

REFERENCES

[1] Docker, “Docker Compose documentation,” Docker Docs. Available: https://docs.docker.com/compose/ (accessed 3 August 2026).

[2] Expo, “Expo SDK reference,” Expo Documentation. Available: https://docs.expo.dev/versions/latest/ (accessed 3 August 2026).

[3] Expo, “Tutorial: Using React Native and Expo,” Expo Documentation. Available: https://docs.expo.dev/tutorial/introduction/ (accessed 3 August 2026).

[4] Flyway, “Flyway documentation,” Redgate Documentation. Available: https://documentation.red-gate.com/flyway (accessed 3 August 2026).

[5] Google, “Gemini API documentation,” Google AI for Developers. Available: https://ai.google.dev/gemini-api/docs (accessed 3 August 2026).

[6] Microsoft, “TypeScript documentation,” Available: https://www.typescriptlang.org/docs/ (accessed 3 August 2026).

[7] Neon, “Neon documentation,” Available: https://neon.com/docs (accessed 3 August 2026).

[8] Next.js, “Next.js Documentation,” Available: https://nextjs.org/docs (accessed 3 August 2026).

[9] OWASP Foundation, “REST Security Cheat Sheet,” OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html (accessed 3 August 2026).

[10] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation: Transactions,” Available: https://www.postgresql.org/docs/16/tutorial-transactions.html (accessed 3 August 2026).

[11] Redis, “Redis documentation,” Available: https://redis.io/docs/latest/ (accessed 3 August 2026).

[12] Spring, “Spring Boot Reference Documentation,” Available: https://docs.spring.io/spring-boot/reference/ (accessed 3 August 2026).

[13] Spring, “Spring Cloud Gateway Reference Documentation,” Available: https://docs.spring.io/spring-cloud-gateway/reference/ (accessed 3 August 2026).

[14] Spring, “Spring Cloud Netflix / Eureka Reference,” Available: https://docs.spring.io/spring-cloud-netflix/reference/ (accessed 3 August 2026).

[15] SmartBiz project repository, source code, migrations, Docker configuration, tests, PROJECT_CONTEXT.md and PROGRESS.md, local academic project, 2026.

APPENDIX A: API SUMMARY

The following table summarises the principal public route groups. Detailed request and response fields remain defined in source DTOs so that the appendix does not reproduce secrets or environment-specific values.

Table A.1: Principal API groups

Route group	Representative operations
/auth	Signup, login, verify/resend email, forgot/reset password, Google OAuth and profile.
/billing	Plans, subscription status, checkout, payments and provider callbacks/webhooks.
/inventory/categories	Category CRUD.
/inventory/products	Paginated CRUD, barcode, low stock, adjust/restock and image operations.
/inventory/suppliers	CRUD, summary, ledger, payments and adjustments.
/inventory/reservations	Create, commit and release stock reservations.
/customers	Paginated CRUD, purchase totals, due amounts and interactions.
/leads	Paginated CRUD, stage updates and conversion.
/sales	Create/import/list/get and today/weekly/trend analytics.
/sales/payments	eSewa settings, create/check/cancel and callback operations.
/ai	Query, insights, insight cards, invoice/voice/file parsing and import sessions.

APPENDIX B: CONFIGURATION CHECKLIST

Configuration values are supplied through environment variables. Real values are intentionally omitted.

Table B.1: Configuration categories

Category	Examples	Handling
Database	AUTH_DB_URL, INVENTORY_DB_URL, CRM_DB_URL, SALES_DB_URL, AI_DB_URL	Use service-specific database/schema; never commit passwords.
Identity	JWT_SECRET, OAuth client ID/secret, verification URL	Rotate secrets and restrict redirect URLs.
Discovery	EUREKA_URL	Use internal service URL in containers.
AI	GEMINI_API_KEY, model URL	Keep key server-side; handle quota and model availability.
Payments	eSewa/Stripe keys, callback URLs, enabled flags	Use UAT/test values until provider approval and production review.
Clients	EXPO_PUBLIC_API_URL, web gateway URL	Point to reachable HTTPS gateway for deployment.

APPENDIX C: TEST EVIDENCE NOTE

Authoritative automated-test run environment: Java 21 with the repository Maven projects. Total discovered @Test methods: 58. Passed: 57. Failed: 1. The failing test is InsightServiceTest.buildInsightCards_returnsRestockSlowStockBundleAndCashFlowSignals in the AI service. Actual card types were SLOW_MOVING_STOCK, BUNDLE_OPPORTUNITY and CASH_FLOW_WARNING; RESTOCK_SOON was expected but absent.

This appendix records the observed result so that future work can reproduce the issue, decide whether the fixture or selection logic is incorrect, implement the change and rerun the complete reactor. No Java 25 tooling failure is counted as an application-test result because Java 21 is the project runtime.